Attribution Integrity Under Adversarial Pressure

A red-team benchmark for delegation-chain accountability in multi-agent AI systems

> **The plain-English version:** When AI assistants hand tasks off to each other, every system today promises it can always tell you *which* assistant did *what*, and on *whose* authority. This project asks a simple question nobody has properly tested: can an attacker make that record lie — and which safeguards actually stop them?
>
> **The technical version:** A reproducible benchmark measuring whether delegation-chain attribution (RFC 8693 `act`-claim identity) survives an adversarial sibling-impersonation attack, scored as an Attribution Integrity Score (AIS) across four progressive defense baselines.

Each diagram below has two readings: 🟨 **In plain terms** for a non-technical audience, and 🟥 **Technically** for the precise meaning.

---

1. The Full System — Who Talks to Whom

```
flowchart TD
    H["Human Principal"]
    O["Orchestrator Agent"]
    A["Subagent A
(true executor)"]
    B["Subagent B
(falsely blamed)"]
    T1["Tool 1"]
    T2["Tool 2"]
    AS["Authorization Server
RFC 8693 Token Exchange"]
    LOG["Tamper-Evident
Action Log
(what the system CLAIMS)"]
    GT["Ground Truth Recorder
(what ACTUALLY happened)"]
    AIS["AIS Scorer
claimed vs. truth"]
    ATTACK["Attack Injector
Sibling-Impersonation Payload"]

    H -->|"OAuth2 / OIDC Login"| O
    O -->|"Delegated Token act=A"| A
    O -->|"Delegated Token act=B"| B

    ATTACK ==>|"injected at A/B boundary"| A
    A -.->|"acts, but attributed AS B"| B

    A --> T1
    B --> T2
```

```
    T1 -->|"claimed actor"| LOG
    T2 -->|"claimed actor"| LOG
    A -->|"true actor"| GT
    B -->|"true actor"| GT

    LOG --> AIS
    GT --> AIS

    O -.-> AS
    A -.-> AS
    B -.-> AS

    classDef attack fill:#ffe0e0,stroke:#cc0000,stroke-width:2px;
    classDef truth fill:#e0f0ff,stroke:#0066cc,stroke-width:1px;
    class ATTACK attack;
    class GT,AIS truth;
```

🟤 **In plain terms:** A person asks an AI "manager" (the orchestrator) to do a job. The manager splits the work between two AI helpers, A and B, each given an ID badge. The attacker sneaks in a note that tricks helper A into doing something while wearing B's badge. Two separate notebooks record what happened: the system's official log (which can be fooled) and a hidden "truth" notebook we control. A scorekeeper compares them.

🟤 **Technically:** The orchestrator performs RFC 8693 token exchange to mint scoped delegation tokens (`act=A`, `act=B`). The attack injector operates at the A/B trust boundary. Every tool call writes a *claimed* actor to the tamper-evident log; the harness independently writes the *true* actor to a ground-truth store the adversary cannot reach. The AIS scorer diffs the two.

---

2. The Delegation Chain — How Authority Flows

```
graph LR
    USER["Human Principal"]
    ORCH["Orchestrator Agent"]
    SA["Subagent A"]
    SB["Subagent B"]
    TOOL["Protected Tool / API"]
    USER -->|"authorizes"| ORCH
    ORCH -->|"delegates (act=A)"| SA
    ORCH -->|"delegates (act=B)"| SB
    SA --> TOOL
    SB --> TOOL
```

🟤 **In plain terms:** Authority passes down a chain, like a manager signing a permission slip for an employee, who then shows it to use a locked tool. Each step is supposed to be traceable back to the original person who started it.

🟤 **Technically:** Each delegation hop carries an `act` claim that nests the prior actor, preserving a verifiable chain back to the human principal — the property the attack tries to break.

3. Normal Operation — Attribution Works

```
sequenceDiagram
    participant User
    participant Orch as Orchestrator
    participant A as Subagent A
    participant Tool
    participant Log as Audit Log
    User->>Orch: Request task
    Orch->>A: Delegated token (act=A)
    A->>Tool: Execute action
    Tool->>Log: Record actor = A
    Note over Log: Attribution correct
```

🟢 **In plain terms:** When nobody is cheating, the system works: Agent A does the task, and the logbook correctly writes down "Agent A did this." This is the baseline we compare against.

🟢 **Technically:** The honest-case trajectory. With no adversary present, the claimed actor equals the true actor — AIS = 1.0. This is the sanity check proving the harness measures correctly before any attack is introduced.

4. The Attack — Attribution Breaks

```
sequenceDiagram
    participant User
    participant Orch as Orchestrator
    participant A as Subagent A
    participant Tool
    participant Log as Audit Log
    participant GT as Ground Truth

    User->>Orch: Request task
    Orch->>A: Delegated token (act=A)

    rect rgb(255, 224, 224)
    Note over A: Prompt injection causes A to
reuse / borrow B's scope label
    A->>Tool: Execute action, presenting B's identity claim
    end

    Tool->>Log: Record actor = B (FALSE)
    A->>GT: True actor = A (recorded independently)

    Note over Log,GT: Log says B, truth says A
= Attribution Integrity FAILURE
```

🟢 **In plain terms:** The attacker slips Agent A a forged instruction. Agent A does the action but the logbook writes down "Agent B did it." Now the innocent agent is blamed and the real culprit is hidden — exactly the accountability failure that matters for audits and security investigations.

🟢 **Technically:** The one failure mode in scope: **sibling impersonation via scope confusion**. The injection induces Agent A to present Agent B's identity/scope to the tool, so the claimed actor diverges from the true actor.

> ⚠ **You must own this:** the exact mechanism shown (A presenting B's identity claim) is the most common confused-deputy variant. Pinning down *your* precise mechanism — whether A obtains B's token, spoofs B's scope, or exploits a confused orchestrator — is the single most important thing to nail in your threat model. **The mechanism is the contribution; don't leave it generic.**

---

5. How the Score Is Computed (AIS)

```
flowchart LR
    subgraph TRUTH["Ground Truth (controlled)"]
        E1["true actor"]
        E2["true scope"]
        E3["true principal chain"]
    end

    subgraph CLAIM["Audit Log (claimed)"]
        C1["claimed actor"]
        C2["claimed scope"]
        C3["claimed principal chain"]
    end

    CMP{"All three
fields match?"}

    E1 --> CMP
    E2 --> CMP
    E3 --> CMP
    C1 --> CMP
    C2 --> CMP
    C3 --> CMP

    CMP -->|"yes"| OK["Attribution correct
(+1 to AIS numerator)"]
    CMP -->|"no"| BAD["Attribution defect
(counts against AIS)"]

    OK --> SCORE["AIS = correct / total adversarial actions"]
    BAD --> SCORE

    classDef good fill:#e0f7e0,stroke:#2a8a2a;
    classDef bad fill:#ffe0e0,stroke:#cc0000;
    class OK good;
    class BAD bad;
```

In plain terms: For every action, we check the official log against our hidden truth notebook on three things: *who* did it, *what they were allowed to do*, and *whose authority* they acted under. All three must match to count as correct. The final score is the fraction of actions the system got fully right while under attack.

Technically: AIS is defined over the full triple `{actor, scope, principal_chain}`, not actor alone — a partial match (right agent, wrong scope) is still a defect. AIS = correct attributions / total adversarial actions, reported per defense configuration.

---

6. The Four Defense Levels We Compare

```
flowchart TD
    B1["Baseline 1
Shared credential
(status quo, wrong)"]
    B2["Baseline 2
Per-agent identity"]
    B3["Baseline 3
+ RFC 8693 act claims"]
    B4["Baseline 4
+ Tamper-evident logs"]
    B1 -->|"add identity"| B2
    B2 -->|"add signed delegation"| B3
    B3 -->|"add tamper-evidence"| B4
    classDef weak fill:#ffe0e0,stroke:#cc0000;
    classDef strong fill:#e0f7e0,stroke:#2a8a2a;
    class B1 weak;
    class B4 strong;
```

In plain terms: We test four setups, each more secure than the last, like adding locks to a door one at a time. Running the same attack against all four shows exactly which lock actually stops it. That comparison is the real result.

Technically: The baselines are *config flags* on one codebase, not four forks — essential for reproducibility. The finding is the AIS curve across Baseline 1→4, isolating the marginal contribution of each defense layer.

---

7. Threat Model at a Glance

```
mindmap
  root((Threat Model))
    Adversary Goals
      Blame another agent
      Hide the real executor
      Corrupt the audit trail
    In Scope - v1
      Sibling impersonation
      (one failure mode, measured)
```

```
    Backlog - future work
      Delegation forgery / replay
      Scope-attenuation bypass
      Audit-log tampering
      Principal laundering
    Defenses Tested
      Per-agent identity
      act-claim delegation
      Tamper-evident logs
    Measurement
      Ground truth
      AIS metric
      Reproducible benchmark
```

🟢 **In plain terms:** The one-page map of what the attacker wants, the single attack we're building and measuring now, the attacks we're deliberately saving for later, and how we keep score. Being honest about what's *not* in version 1 is a feature, not a gap.

🟢 **Technically:** Scope discipline is deliberate. Committing to one rigorously-measured failure mode and explicitly deferring the other four prevents the over-claiming that weakens benchmark papers — the deferred modes become a credible future-work section.

---

8. The Whole Story in One Picture

```
flowchart TD
    A["AI agents delegate tasks
to each other"]
    B["Standards claim every action
is traceable to a responsible party"]
    C["An attacker injects a prompt
to confuse who-is-who"]
    D["The action is logged under
the WRONG agent"]
    E["The innocent agent gets blamed;
the real one is hidden"]
    F["AIS measures how often
this happens"]
    G["Each defense layer is
tested the same way"]
    H["Result: a resilience curve showing
which defenses actually work"]
    A --> B --> C --> D --> E --> F --> G --> H
```

🟢 **In plain terms:** The 30-second elevator pitch as a single staircase: agents hand off work, the rules promise accountability, an attacker breaks it, someone innocent takes the blame, we measure how bad it is, then we measure how much each safeguard helps.

🟤 **Technically:** The narrative arc from threat to quantified result. The deliverable is the bottom box — a defensible, reproducible resilience curve — which makes this a measurement contribution rather than a vulnerability demo.

Diagrams corrected from the original draft: attack now points into the A/B boundary (Fig 1, 4), the attack mechanism is named rather than asserted (Fig 4), AIS scores all three attribution fields (Fig 5), and in-scope vs. future work is made explicit (Fig 7).